

InfoWorld DeepDive

17 ways to speed your SQL queries



Deep Dive

17 ways to speed your SQL queries

It's easy to create database code that slows down query results or ties up the database unnecessarily -- unless you follow these tips.

BY SEAN MCCOWN

SQL developers on every platform are struggling, seemingly stuck in a DO WHILE loop that makes them repeat the same mistakes again and again. That's because the database field is still relatively immature. Sure, vendors are making some strides, but they continue to grapple with the bigger issues. Concurrency, resource management, space management, and speed still plague SQL developers whether they're coding on SQL Server, Oracle, DB2, Sybase, [MySQL](#), or any other relational platform.

Part of the problem is that there is no magic bullet, and for almost every best practice, I can show you at least one exception. Typically, a developer finds his or her own favorite methods -- though usually they don't include any constructs for performance or concurrency -- and doesn't bother exploring other options.

Maybe that's a symptom of lack of education, or the developers are just too close to the process to recognize when they're doing something wrong. Maybe the query runs well on a local set of test data but fails miserably on the production system.

I don't expect SQL developers to become administrators, but they must take production issues into account when writing their code. If they don't do it during initial development, the DBAs will just make them go back and do it later -- and the users suffer in the interim.

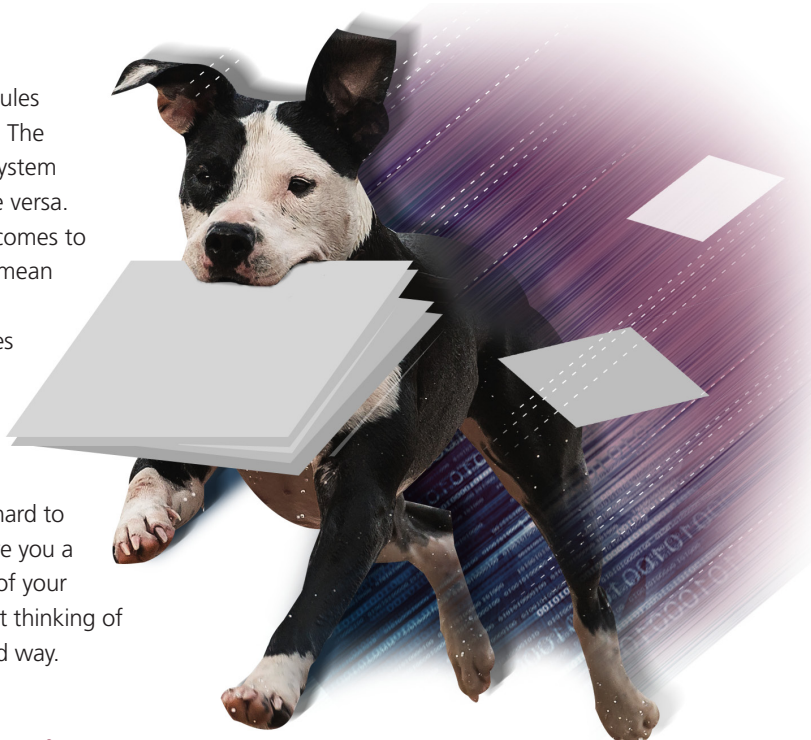
There's a reason why we say tuning a database is both an art and a science. It's



Deep Dive

because very few hard-and-fast rules exist that apply across the board. The problems you've solved on one system aren't issues on another, and vice versa. There's no right answer when it comes to tuning queries, but that doesn't mean you should give up.

There are some good principles you can follow that should yield results in one combination or another. I've encapsulated them in a list of SQL dos and don'ts that often get overlooked or are hard to spot. These techniques should give you a little more insight into the minds of your DBAs, as well as the ability to start thinking of processes in a production-oriented way.



1 Don't use UPDATE instead of CASE

This issue is very common, and though it's not hard to spot, many developers often overlook it because using UPDATE has a natural flow that seems logical.

Take this scenario, for instance: You're inserting data into a temp table and need it to display a certain value if another value exists. Maybe you're pulling from the Customer table and you want anyone with more than \$100,000 in orders to be labeled as "Preferred." Thus, you insert the data into the table and run an UPDATE statement to set the CustomerRank column to "Preferred" for anyone who has more than \$100,000 in orders. The problem is that the UPDATE statement is logged, which means it has to write twice for every single write to the table. The way around this, of course, is to use an inline CASE statement in the SQL query itself. This tests every row for the order amount condition and sets the "Preferred" label before it's written to the table. The performance increase can be staggering.

2 Don't blindly reuse code

This issue is also very common. It's very easy to

copy someone else's code because you know it pulls the data you need. The problem is that quite often it pulls much more data than you need, and developers rarely bother trimming it down, so they end up with a huge superset of data. This usually comes in the form of an extra outer join or an extra condition in the WHERE clause. You can get huge performance gains if you trim reused code to your exact needs.

3 Do pull only the number of columns you need

This issue is similar to issue No. 2, but it's specific to columns. It's all too easy to code all your queries with `SELECT *` instead of listing the columns individually. The problem again is that it pulls more data than you need. I've seen this error dozens and dozens of times. A developer does a `SELECT *` query against a table with 120 columns and millions of rows, but winds up using only three to five of them. At that point, you're processing so much more data than you need it's a wonder the query returns at all. You're not only processing more data than you need, but you're also taking resources away from other processes.



These techniques should give you a little more insight into the minds of your DBAs, as well as the ability to start thinking of processes in a production-oriented way.

Deep Dive

4 Don't double-dip

Here's another one I've seen more times than I should have: A stored procedure is written to pull data from a table with hundreds of millions of rows. The developer needs customers who live in California and have incomes of more than \$40,000. So he queries for customers that live in California and puts the results into a temp table; then he queries for customers with incomes above \$40,000 and puts those results into another temp table. Finally, he joins both tables to get the final product.

Are you kidding me? This should be done in a single query; instead, you're double-dipping a superlarge table. Don't be a moron: Query large tables only once whenever possible -- you'll find how much better your procedures perform.

A slightly different scenario is when a subset of a large table is needed by several steps in a process, which causes the large table to be queried each time. Avoid this by querying for the subset and persisting it elsewhere, then pointing the subsequent steps to your smaller data set.



Don't be a moron: Query large tables only once whenever possible -- you'll find how much better your procedures perform.

5 Do know when to use temp tables

This issue is a bit harder to get a handle on, but it can yield impressive gains. You can use temp tables in a number of situations, such as keeping you from double-dipping into large tables. You can also use them to greatly decrease the processing power required to join large tables. If you must join a table to a large table and there's a condition on that large table, you can improve performance by pulling out the subset of data you need from the large table into a temp table and joining with that instead. This is also helpful (again) if you have several queries in the procedure that have to make similar joins to the same table.

6 Do pre-stage data

This is one of my favorite topics because it's an old technique that's often overlooked. If you have a report or a procedure (or better yet, a set of them) that will do similar joins to large tables, it can be a benefit for you to pre-stage the data by joining the tables ahead of time and persisting them into a table. Now the reports can run against that pre-staged table and avoid the large join.

You're not always able to use this technique, but when you can, you'll find it is an excellent way to save server resources.

Note that many developers get around this join problem by concentrating on the query itself and creating a view-only around the join so that they don't have to type the join conditions again and again. But the problem with this approach is that the query still runs for every report that needs it. By pre-staging the data, you run the join just once (say, 10 minutes before the reports) and everyone else avoids the big join. I can't tell you how much I love this technique; in most environments, there are popular tables that get joined all the time, so there's no reason why they can't be pre-staged.



Deep Dive



Along these lines, many developers have it stuck in their heads that these delete and update operations must be completed the same day. That's not always true, especially if you're archiving.

7 Do delete and update in batches

Here's another easy technique that gets overlooked a lot. Deleting or updating large amounts of data from huge tables can be a nightmare if you don't do it right. The problem is that both of these statements run as a single transaction, and if you need to kill them or if something happens to the system while they're working, the system has to roll back the entire transaction. This can take a very long time. These operations can also block other transactions for their duration, essentially bottlenecking the system.

The solution is to do deletes or updates in smaller batches. This solves your problem in a couple ways. First, if the transaction gets killed for whatever reason, it only has a small number of rows to roll back, so the database returns online much quicker. Second, while the smaller batches are committing to disk, others can sneak in and do some work, so concurrency is greatly enhanced.

Along these lines, many developers have it stuck in their heads that these delete and update operations must be completed the same day. That's not always true, especially if you're archiving. You can stretch that operation out as long as you need to, and the smaller batches help accomplish that. If you can take longer to do these intensive operations, spend the extra time and don't bring your system down.

8 Do use temp tables to improve cursor performance

I hope we all know by now that it's best to stay away from cursors if at all possible. Cursors not only suffer from speed problems, which in itself can be an issue with many operations, but they can also cause your operation to block



other operations for a lot longer than is necessary. This greatly decreases concurrency in your system.

However, you can't always avoid using cursors, and when those times arise, you may be able to get away from cursor-induced performance issues by doing the cursor operations against a temp table instead. Take, for example, a cursor that goes through a table and updates a couple of columns based on some comparison results. Instead of doing the comparison against the live table, you may be able to put that data into a temp table and do the comparison against that instead. Then you have a single UPDATE statement against the live table that's much smaller and holds locks only for a short time.

Sniping your data modifications like this can greatly increase concurrency. I'll finish by saying you almost never need to use a cursor. There's almost always a set-based solution; you need to learn to see it.

9 Don't nest views

Views can be convenient, but you need to be careful when using them. While views can help to obscure large queries from users and to standardize data access, you can easily find yourself in a situation where you have views that call views that call views that call views. This is called nesting views, and it can cause severe

Deep Dive



This is one of my favorite tricks of all time because it is truly one of those hidden secrets that only the experts know. When you use a scalar function in the `SELECT` list of a query, the function gets called for every single row in the result set.

performance issues, particularly in two ways. First, you will very likely have much more data coming back than you need. Second, the query optimizer will give up and return a bad query plan.

I once had a client that loved nesting views. The client had one view it used for almost everything because it had two important joins. The problem was that the view returned a column with 2MB documents in it. Some of the documents were even larger. The client was pushing at least an extra 2MB across the network for every single row in almost every single query it ran. Naturally, query performance was abysmal.

And none of the queries actually used that column! Of course, the column was buried seven views deep, so even finding it was difficult. When I removed the document column from the view, the time for the biggest query went from 2.5 hours to 10 minutes. When I finally unraveled the nested views, which had several unnecessary joins and columns, and wrote a plain query, the time for that same query dropped to subseconds.

10 Do use table-valued functions

This is one of my favorite tricks of all time because it is truly one of those hidden secrets that only the experts know. When you use a scalar function in the `SELECT` list of a query, the function gets called for every single row in the result set. This can reduce the performance of large queries by a significant amount. However, you can greatly improve the performance by converting the scalar function to a table-valued function and using a `CROSS APPLY` in the query. This is a wonderful trick that can yield great improvements.

Want to know more about the `APPLY` operator? You'll find a full discussion in an excellent course on [Microsoft Virtual Academy](#) by Itzik Ben-Gan.

11 Do use partitioning to avoid large data moves

Not everyone will be able to take advantage of this tip, which relies on partitioning in SQL Server Enterprise, but for those of you who can, it's a great trick. Most people don't realize that all tables in SQL Server are partitioned. You can separate a table into multiple partitions if you like, but even simple tables are partitioned from the time they're created; however, they're created as single partitions. If you're running SQL Server Enterprise, you already have the advantages of partitioned tables at your disposal.

This means you can use partitioning features like `SWITCH` to archive large amounts of data from a warehousing load. Let's look at a real example from a client I had last year. The client had the requirement to copy the data from the current day's table into an archive table; in case the load failed, the company could quickly recover with the current day's table. For various reasons, it couldn't rename the tables back and forth every time, so the company inserted the data into an archive table every day before the load, then deleted the current day's data from the live table.

This process worked fine in the beginning, but a year later, it was taking 1.5 hours to copy each table -- and several tables had to be copied every day. The problem was only going to get worse. The solution was to scrap the `INSERT` and `DELETE` process and use the `SWITCH` command. The `SWITCH` command allowed the company to avoid all of the writes because it assigned the pages to the archive table. It's only a metadata change. The `SWITCH` took on average between two and three seconds to run. If the current load ever fails, you `SWITCH` the data back into the original table.

This is a case where understanding that all tables are partitions slashed hours from a data load.

12 If you must use ORMs, use stored procedures

This is one of my regular diatribes. In short, don't use ORMs (object-relational mappers). ORMs produce some of the worst code on the planet, and they're responsible for almost every performance issue I get involved in. ORM code genera-

Deep Dive



This is one of my regular diatribes. In short, don't use ORMs (object-relational mappers). **ORMs produce some of the worst code on the planet, and they're responsible for almost every performance issue I get involved in.**

tors can't possibly write SQL as well as a person who knows what they're doing. However, if you use an ORM, write your own stored procedures and have the ORM call the stored procedure instead of writing its own queries. Look, I know all the arguments, and I know that developers and managers love ORMs because they speed you to market. But the cost is incredibly high when you see what the queries do to your database.

Stored procedures have a number of advantages. For starters, you're pushing much less data across the network. If you have a long query, then it could take three or four round trips across the network to get the entire query to the database server. That's not including the time it takes the server to put the query back together and run it, or considering that the query may run several -- or several hundred -- times a second.

Using a stored procedure will greatly reduce that traffic because the stored procedure call will always be much shorter. Also, stored procedures are easier to trace in Profiler or any other tool. A stored procedure is an actual object in your database. That means it's much easier to get performance statistics on a stored procedure than on an ad-hoc query and, in turn, find performance issues and draw out anomalies.

In addition, stored procedures parameterize more consistently. This means you're more likely to reuse your execution plans and even deal with caching issues, which can be difficult to pin down with ad-hoc queries. Stored procedures also make it much easier to deal with edge cases and even add auditing or change-locking behavior. A stored procedure can handle many tasks that trouble ad-hoc queries. My wife unraveled a two-page query from Entity Framework a couple of years ago. It took 25 minutes to run. When she boiled it down to its essence, she rewrote that huge query as `SELECT COUNT(*) FROM T1`. No kidding.

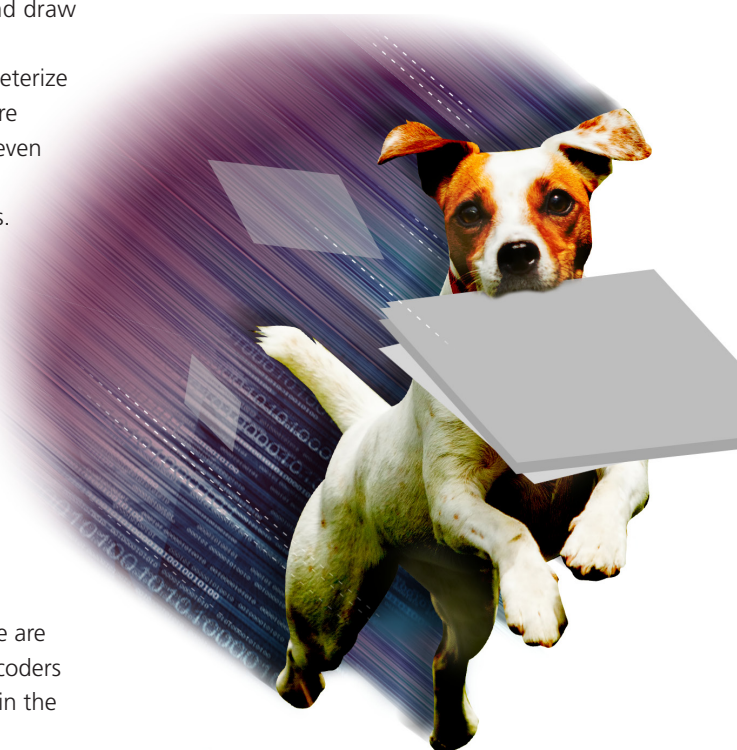
OK, I kept it as short as I could. Those are the high-level points. I know many .Net coders think that business logic doesn't belong in the

database, but what can I say other than you're outright wrong. By putting the business logic on the front end of the application, you have to bring all of the data across the wire merely to compare it. That's not good performance. I had a client earlier this year that kept all of the logic out of the database and did everything on the front end. The company was shipping hundreds of thousands of rows of data to the front end, so it could apply the business logic and present the data it needed. It took 40 minutes to do that. I put a stored procedure on the back end and had it call from the front end; the page loaded in three seconds.

Of course, the truth is that sometimes the logic belongs on the front end and sometimes it belongs in the database. But ORMs always get me ranting.

13 Don't do large ops on many tables in the same batch

This one seems obvious, but apparently it's not. I'll use another live example because it will drive home the point much better. I had a system that suffered tons of blocking. Dozens of operations were at a standstill. As it turned out, a



Deep Dive

delete routine that ran several times a day was deleting data out of 14 tables in an explicit transaction. Handling all 14 tables in one transaction meant that the locks were held on every single table until all of the deletes were finished. The solution was to break up each table's deletes into separate transactions so that each delete transaction held locks on only one table. This freed up the other tables and reduced the blocking and allowed other operations to continue working. You always want to split up large transactions like this into separate smaller ones to prevent blocking.



Don't use triggers unless it's unavoidable -- and it's almost always avoidable.

14 **Don't use triggers**

This one is largely the same as the previous one, but it bears mentioning. Don't use triggers unless it's unavoidable -- and it's almost always avoidable.

The problem with triggers: Whatever it is you want them to do will be done in the same transaction as the original operation. If you write a trigger to insert data into another table when you update a row in the Orders table, the lock will be held on both tables until the trigger is done. If you need to insert data into another table after the update, then put the update and the insert into a stored procedure and do them in separate transactions. If you need to roll back, you can do so easily without having to hold locks on both tables. As always, keep transactions as short as possible and don't hold locks on more than one resource at a time if you can help it.

15 **Don't cluster on GUID**

After all these years, I can't believe we're still fighting this issue. But I still run into clustered GUIDs at least twice a year.

A GUID (globally unique identifier) is a 16-byte randomly generated number. Ordering your table's data on this column will cause your table to fragment much faster than using a steadily increasing value like DATE or IDENTITY. I did a benchmark a few years ago where I inserted a

bunch of data into one table with a clustered GUID and into another table with an IDENTITY column. The GUID table fragmented so severely that the performance degraded by several thousand percent in a mere 15 minutes. The IDENTITY table lost only a few percent off performance after five hours. This applies to more than GUIDs -- it goes toward any volatile column.

16 **Don't count all rows if you only need to see if data exists**

It's a common situation. You need to see if data exists in a table or for a customer, and based on the results of that check, you're going to perform some action. I can't tell you how often I've seen someone do a `SELECT COUNT(*) FROM dbo.T1` to check for the existence of that data:

```
SET @CT = (SELECT COUNT(*) FROM
dbo.T1);
If @CT > 0
BEGIN
<Do something>
END
```

It's completely unnecessary. If you want to check for existence, then do this:

```
If EXISTS (SELECT 1 FROM dbo.T1)
BEGIN
<Do something>
END
```

Don't count everything in the table. Just get back the first row you find. SQL Server is smart enough to use EXISTS properly, and the second block of code returns superfast. The larger the table, the bigger difference this will make. Do the smart thing now before your data gets too big. It's never too early to tune your database.

In fact, I just ran this example on one of my production databases against a table with 270 million rows. The first query took 15 seconds, and included 456,197 logical reads, while the second one returned in less than one second and included only five logical reads. However, if you

Deep Dive



really do need a row count on the table, and it's really big, another technique is to pull it from the system table. `SELECT` rows from `sysindexes` will get you the row counts for all of the indexes. And because the clustered index represents the data itself, you can get the table rows by adding `WHERE indid = 1`. Then simply include the table name and you're golden. So the final query is `SELECT` rows from `sysindexes` where `object_name(id) = 'T1'` and `indexid = 1`. In my 270 million row table, this returned sub-second and had only six logical reads. Now that's performance.

17 Don't do negative searches

Take the simple query `SELECT * FROM Customers WHERE RegionID <> 3`. You can't use an index with this query because it's a negative search that has to be compared row by row with a table scan. If you need to do something like this, you may find it performs much better if you rewrite the query to use the index.

This query can easily be rewritten like this:

```
SELECT * FROM Customers WHERE
RegionID < 3 UNION ALL SELECT *
FROM Customers WHERE RegionID
```

This query will use an index, so if your data set is large it could greatly outperform the table scan version. Of course, nothing is ever that easy, right? It could also perform worse, so test this before you implement it. There are too many factors involved for me to tell you that it will work 100 percent of the time. Finally, I realize this query breaks the No. 4 rule, "No Double Dipping," but that goes to show there are no hard and fast rules. Though we're double dipping here, we're doing it to avoid a costly table scan.

OK, there you go. You won't be able to apply all of these tips all of the time, but if you keep them in mind you'll find yourself using them as solutions to some of your biggest issues. The most important thing to remember is not to take anything I say as the gospel and implement it because I said so. Test everything in your environment, then test it again. The same solutions won't work in every situation. But these are tactics I use all the time when addressing poor performance, and they have all served me well time and again. ■

Sean McCown is a *Certified Master in SQL Server* and a *SQL Server MVP* with 20 years of experience in databases.